



POLITECNICO
MILANO 1863

RELAZIONE
PROVA FINALE DI RETI LOGICHE

a cura di

Loreto Del Vecchio - 10856752

Giuseppe D'Ambrosi - 10848505

INDICE

1.INTRODUZIONE.....	1
a. COMANDI.....	2
b. FUNZIONE DEL FILTRO.....	2
c. FUNZIONAMENTO DEL FILTRO: VALORI AI BORDI.....	3
d. FUNZIONAMENTO DEL FILTRO: DIVISIONI.....	3
e. SEGNALI DI CLOCK E RESET.....	3
2.INTRODUZIONE.....	4
a. SCHEMA DI ARCHITETTURA.....	4
b. FUNZIONAMENTO GENERALE.....	5
c. FSM - DIRETTORE DELLE OPERAZIONI.....	6
i. FSM: DESCRIZIONE DEGLI STATI.....	8
ii. FSM: DESCRIZIONE DEI SEGNALI.....	9
d. SHIFT REGISTER.....	10
i. SHIFT REGISTER: DESCRIZIONE DEI SEGNALI.....	10
e. CONTATORE.....	11
i. CONTATORE : DESCRIZIONE DEI SEGNALI.....	11
f. BUFFER.....	11
i. BUFFER: DESCRIZIONE DEI SEGNALI.....	11
g. FILTRO DIFFERENZIALE.....	12
i. FILTRO DIFFERENZIALE: DESCRIZIONE DEI SEGNALI.....	12
h. ANALISI DELLE FASI DI PROGETTAZIONE.....	13
3. RISULTATI SPERIMENTALI.....	14
a. SINTESI.....	14
i. ANALISI DEL TIMING REPORT.....	14
ii. ANALISI DELLA LOGICA.....	15
b. SIMULAZIONI.....	16
i. TEST FORNITI DAL DOCENTE.....	16
ii. MINIMO NUMERO DI DATI IN INPUT.....	16
iii. MASSIMO NUMERO DI DATI IN INPUT.....	17
iv. MASSIMO NUMERO DI DATI IN INPUT +1.....	17
v. 0 PRE E POST NORMALIZZAZIONE.....	17
vi. TUTTI 0.....	18
vii. COSTANTE.....	18
viii. VALORE MASSIMO E MINIMO IN INPUT.....	18
ix. START MULTIPLI.....	19
x. RESET.....	19
xi. TEST DI ROBUSTEZZA E DI CORRETTEZZA DI ACCESSO ALLA MEMORIA.....	19
xii. TEST CON COEFFICIENTI C1- C14 CASUALI.....	19
xiii. START E RESET COMBINATI.....	19
c. CONCLUSIONI DELLA FASE DI SIMULAZIONE.....	20
4.CONCLUSIONI.....	21

1.INTRODUZIONE

L'obiettivo del seguente progetto è quello di **descrivere e sintetizzare un componente HW** che si occupa di interfacciarsi con una memoria esterna, leggerne comandi e dati, applicare un filtro differenziale ai dati letti seguendo le modalità specificate nei comandi iniziali, e scrivere i risultati all'interno della memoria stessa.

La **entity** del componente, che ne descrive i segnali di ingresso e uscita, è la seguente:

```
entity project_reti_logiche is
  port(
    i_clk: in std_logic;
    i_rst: in std_logic;
    i_start: in std_logic;
    i_add: in std_logic_vector(15 downto 0);

    o_done: out std_logic;

    o_mem_addr: out std_logic_vector(15 downto 0);
    i_mem_data: in std_logic_vector(7 downto 0);
    o_mem_data: out std_logic_vector(7 downto 0);
    o_mem_we: out std_logic;
    o_mem_en: out std_logic
  );
end entity;
```

Si noti che **tutti i segnali sono sincroni al clock**, a eccezione del RESET.

È inoltre garantito che prima del segnale di START venga eseguito il RESET di tutti i segnali.

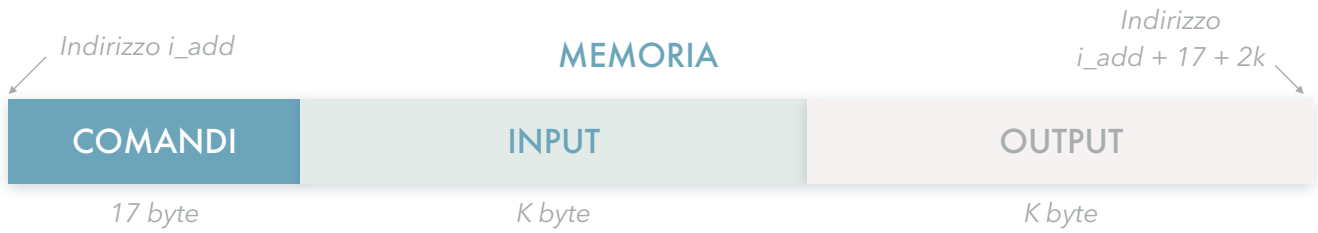
- *i_start* segnala l'inizio delle operazioni;
- *i_add* indica l'indirizzo da cui partire a leggere i valori in memoria;
- *o_done* viene posto a '1' dal componente una volta terminate tutte le operazioni.

La restante parte dei segnali serve ad interagire con la memoria. Dal momento in cui questa è abilitata tramite *o_mem_en*, viene indirizzata all'indirizzo contenuto in *o_mem_addr* e, se *o_mem_we* è abilitata, scrive a quell'indirizzo i dati contenuti in *o_mem_data*, altrimenti mette a disposizione i dati di quell'indirizzo sul segnale *i_mem_data*. Segue la memoria vista come entità:



1.INTRODUZIONE

Segue una spiegazione della struttura della memoria come architettura:



a. COMANDI

Dei 17 byte di comandi:

- i primi due byte sono **k1** e **k2** che rappresentano la lunghezza k della sequenza di parole da un byte da leggere in memoria ($k1$ più significativo);
- Il terzo byte è **S** e il suo bit meno significativo permette di capire il filtro da applicare (0 per il filtro modulo 3 e 1 per il filtro modulo 5);
- Gli ultimi 14 byte, da **C1** a **C14**, memorizzano i coefficienti dei due filtri.

I coefficienti dei filtri potrebbero essere, ad esempio:

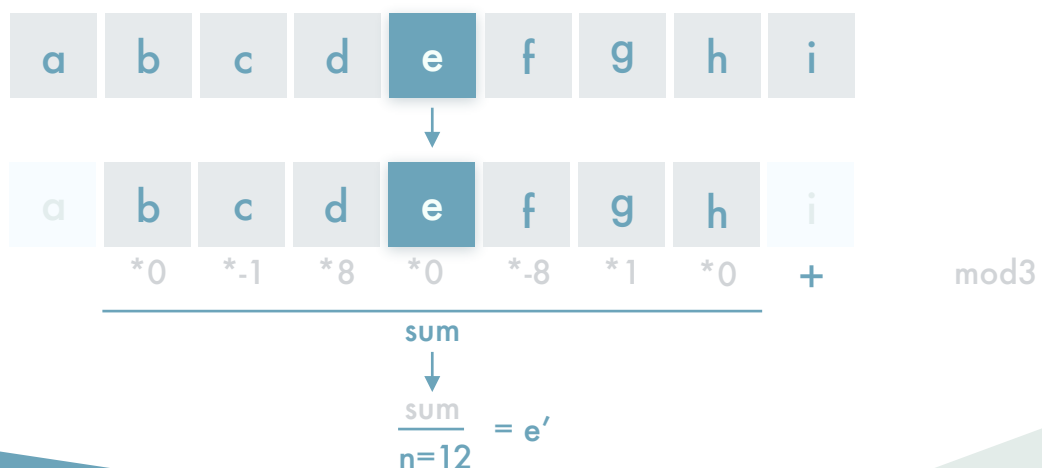
- ordine 3: 0, -1, 8, 0, -8, 1, 0;
- ordine 5: 1, -9, -45, 0, -45, 9, -1.

Inoltre, per ciascuno dei due filtri, è noto un termine di normalizzazione **n: 12 (ord.3) e 60 (ord. 5)**.

I coefficienti precedentemente elencati possono variare, per cui devono esser letti a inizio di ogni elaborazione.

b. FUNZIONE DEL FILTRO

Per ogni elemento letto, il filtro moltiplica l'elemento in considerazione, i tre che lo precedono e i tre che lo seguono, ciascuno per un coefficiente (i coefficienti variano al variare del modulo e sono utilizzati in ordine, dall'elemento con la posizione minore a quello con la posizione maggiore), poi effettua la somma dei risultati e divide per il fattore di normalizzazione corrispondente al filtro che si sta utilizzando. Segue uno schema esplicativo col mod3 che emula il calcolo per l'elemento centrale (il calcolo ovviamente nella realtà va fatto anche con tutti gli altri elementi):



1.INTRODUZIONE

Lo schema alla pagina precedente illustra solo il caso generale per il modulo 3, il cui risultato andrà posizionato in memoria all'indirizzo corrispondente (posizione di $e + k$).

Possiamo riassumere il comportamento della macchina con la seguente formula matematica:

$$f'(i) = \frac{1}{n} \sum_{j=-l}^{+l} C_j \cdot f[j + i]$$

con l che vale 2 per filtro di ordine 3 e 3 per filtro di ordine 5. C è il coefficiente in posizione j in memoria.

c. FUNZIONAMENTO DEL FILTRO: VALORI AI BORDI

Per i valori ai bordi (i primi tre e gli ultimi tre della sequenza da convertire), in cui mancano elementi precedenti o successivi, si utilizzano zeri per riempire tutte le posizioni mancanti. Per il primo elemento, ad esempio, si considereranno tre 0 a rappresentare le tre parole precedenti; per il secondo elemento, che ha un precedente, verranno utilizzati solo due 0 e così via.

d. FUNZIONAMENTO DEL FILTRO: DIVISIONI

Per effettuare le divisioni si ricorda che ogni shift a destra equivale a una divisione per due e produce un troncamento verso -1 per numeri negativi e verso 0 per numeri positivi, ragion per cui al termine delle operazioni si somma 1 per ogni shift se il valore da normalizzare è negativo così da correggere l'errore introdotto tramite troncamento.

e. SEGNALI DI CLOCK E RESET

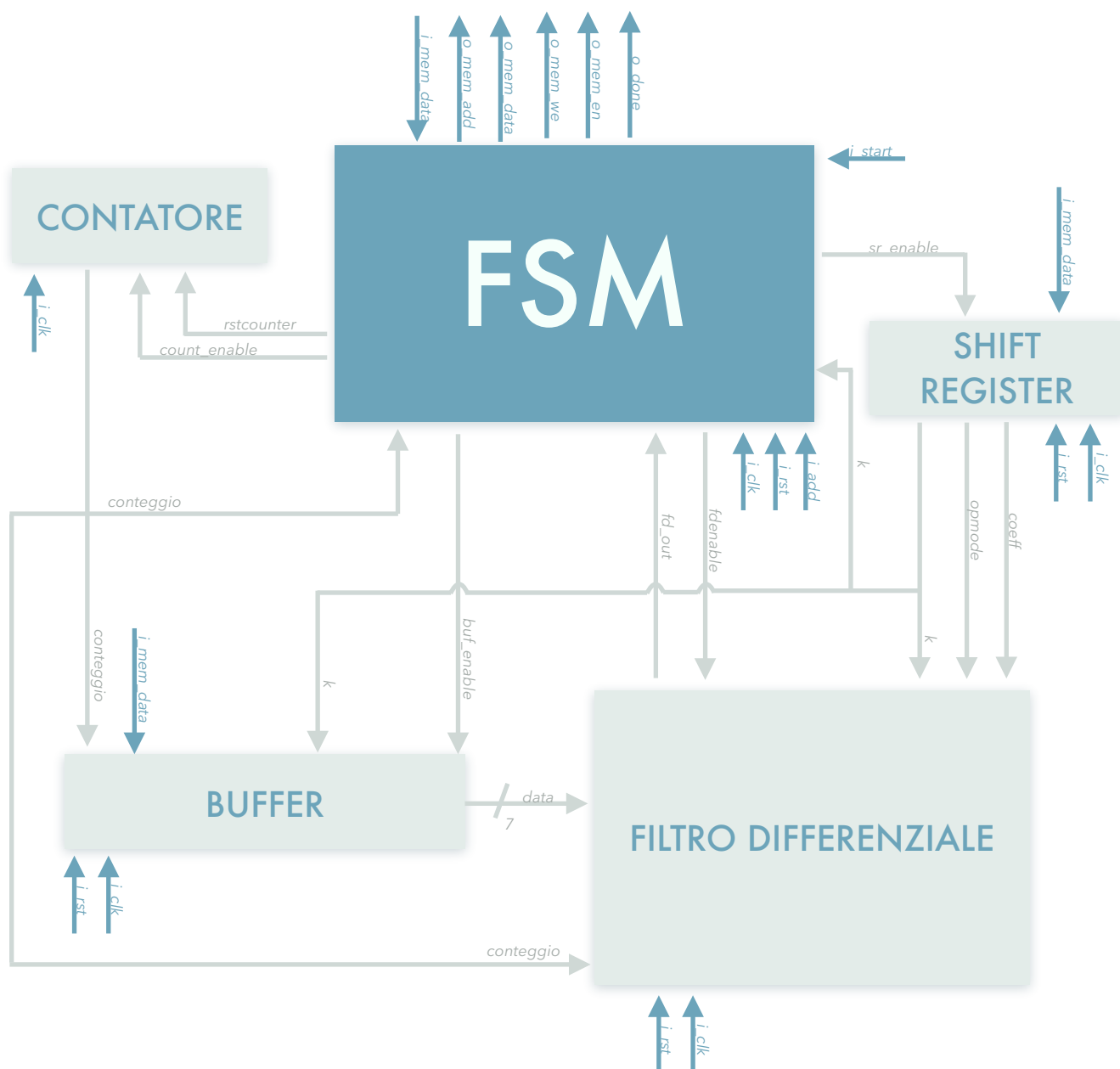
È garantito che all'inizio delle operazioni svolte col componente HW realizzato sarà attivato un segnale di RESET, il quale re-inizializza il modulo. I segnali di START servono ad avviare le elaborazioni e restano alti fin quando il segnale di DONE non viene alzato, quando ciò avviene, il segnale di START viene posto a 0 e conseguentemente anche il segnale di DONE, pronti per ripartire con una nuova elaborazione (all'arrivo di un nuovo segnale di START). I segnali di RESET, invece, possono essere alzati in ogni momento e non è garantito che per i segnali di START successivi al primo ci siano sempre segnali di RESET precedenti.

2.ARCHITETTURA

a. SCHEMA DI ARCHITETTURA

In **blu** sono riportati i segnali primari, quelli definiti nella entity del progetto.

In **verde**, vi sono segnali interni, utilizzati per la comunicazione tra le varie componenti del modulo HW.



2.ARCHITETTURA

Lo schema presente nella pagina precedente illustra ad alto livello il progetto del modulo HW.

Per realizzarlo sono stati utilizzati 5 componenti:

1. FSM;
2. SHIFT REGISTER;
3. CONTATORE;
4. BUFFER;
5. FILTRO DIFFERENZIALE.

b. FUNZIONAMENTO GENERALE

Inizialmente, ci si trova in uno stato di RESET, in cui la memoria è disabilitata e tutti i segnali dei componenti (descritti successivamente in dettaglio per ciascun componente), vengono posti al loro valore di default.

La FSM si occupa di gestire tutte le operazioni e dividere in varie fasi le elaborazioni del modulo HW, ciò consente di modularizzare e rendere più agevole la manutenzione del componente realizzato.

Quando lo START raggiunge '1', il contatore inizia a contare i cicli di clock per cui viene abilitato, così da avere il numero di operazioni svolte e permettere all'FSM di orientarsi meglio nelle fasi prestabilite.

Le istruzioni vengono salvate all'interno dello shift register, mentre i k dati da convertire sono memorizzati all'interno di un buffer. Il salvataggio all'interno del buffer non è necessario in quanto l'FSM ha già il potere di accedere in ogni istante ai dati desiderati nella memoria, tuttavia tale scelta implementativa nasce dal desiderio di suddividere le funzionalità e i compiti per i diversi componenti, semplificando la logica di gestione dell'FSM e avendo una porzione di progetto riservata alla gestione dei dati da elaborare.

Sono state inoltre condotte simulazioni per verificare che tale scelta non andasse ad inficiare sulle performance della macchina in maniera considerevole, effettuando un bilancio costo-convenienza.

Terminata la fase iniziale di immagazzinamento dei dati, il contatore riceverà un segnale di RESET, così da cominciare nuovamente a scorrere tra i k elementi da convertire e, per ogni elemento (dunque, per ogni step indicizzato dal contatore) il buffer si occuperà di esporre al filtro differenziale 7 byte, scorrendo come una finestra mobile.

Al quarto byte ci sarà sempre l'elemento centrale a cui il contatore sta facendo riferimento, dal primo al terzo i suoi tre precedenti e dal quinto al settimo i 3 successivi. Il buffer si occupa anche di aggiungere 0 all'interno delle parole appartenenti alla finestra di uscita, laddove dovessero mancare dati precedenti o successivi.

Ad ogni finestra di 7 byte ricevuta, il filtro differenziale elabora il risultato e lo trasferisce alla FSM, che si occupa di restituirlo alla posizione corretta in memoria, utilizzando il risultato del contatore sommato a k e a 17 per i coefficienti delle istruzioni.

Tutto ciò accade fin quando non termina l'elaborazione dei dati, a quel punto la FSM alza il segnale di DONE e ci si porta in un nuovo stato, in attesa che lo START vada a 0 e, eventualmente torni a 1 per nuove elaborazioni. In ogni momento, l'arrivo di un RESET porta il sistema a ripartire dallo stato di inizio re-inizializzando ogni segnale.

c. FSM - DIRETTORE DELLE OPERAZIONI

Per far funzionare il sistema in modo coordinato, è necessaria una logica di controllo che stabilisca quando eseguire le operazioni di lettura, elaborazione e scrittura. Questo compito è affidato alla **FSM (macchina a stati finiti)**, che gestisce il flusso dell'elaborazione attraverso una sequenza di stati.

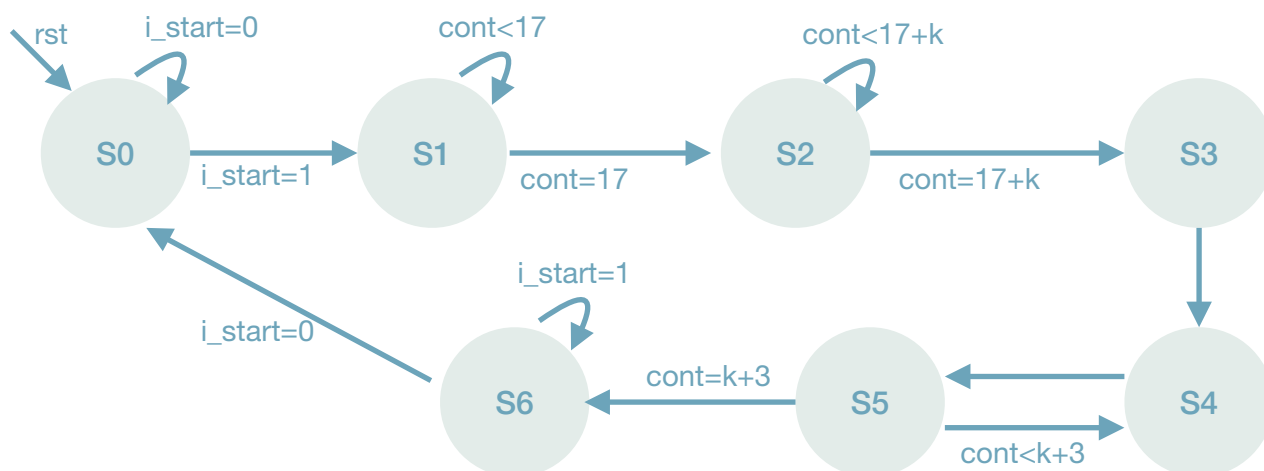
Inizialmente, la FSM attende il segnale di **START**, che indica l'inizio di una nuova elaborazione. A questo punto, il sistema entra in azione seguendo i seguenti passaggi:

1. **Lettura dei parametri iniziali:** nei primi cicli, vengono caricati dalla memoria i dati di configurazione, come la lunghezza della sequenza e i coefficienti del filtro;
2. **Lettura della sequenza da elaborare:** i k dati da convertire vengono letti uno alla volta e memorizzati nel buffer;
3. **Attivazione del filtro:** una volta raccolti i k valori, la FSM avvia l'elaborazione del filtro differenziale;
4. **Scrittura del risultato in memoria:** ad ogni valore elaborato ricevuto, la FSM lo restituisce all'indice corretto in memoria;
5. **Segnalazione del completamento:** quando tutta la sequenza è stata elaborata, la FSM attiva il segnale **DONE**, indicando che il processo è terminato.

Grazie a questa logica di controllo, il sistema esegue tutte le operazioni nell'ordine corretto, garantendo che ogni componente venga attivato solo quando necessario.

Di seguito è illustrato il diagramma degli stati in forma grafica e tabellare, per esplicitare il funzionamento della FSM in modo organico e descrittivo.

Per semplicità grafica, il grafo degli stati rappresenta solo le transizioni, la tabella degli stati invece esplicita in modo corretto e completo le uscite della FSM al variare dello stato.



STATO	RST- COUNTER	COUNT_ ENABLE	SR_ ENABLE	O_MEM_AD DR	O_MEM_ DATA	O_MEM_WE	O_MEM_EN	O_DONE	FD_ ENABLE	BUF_ ENABLE	CONDIZIONE DI USCITA	STATO PROSSIMO
S0 - INIZIO	1	0	0	0	0	0	0	0	0	0	L_START = 1 & L_RST = 0	S1
S1 - LETTURA COMANDI	0	1	1	L_ADD + conteggio	0	0	1	0	0	0	conteggio = 17	S2
S2 - LETTURA BUFFER	0	1	0	L_ADD + conteggio	0	0	1	0	0	1	conteggio = 17 + k	S3
S3 - RESET CONTATORE E PREPARAZIONE FILTRO	1	0	0	0	0	0	0	0	0	0	Dopo un ciclo	S4
S4 - ESECUZIONE FILTRO	0	0	0	0	0	0	0	0	1	0	Dopo un ciclo	S5
S5 - IMMAGAZZINO IN MEMORIA	0	1	0	L_ADD + 17 + k + conteggio - 4	FD_OUT	1 se counter >= 4, altrimenti 0	1	0	0	0	Dopo un ciclo	Conteggio < k+3 => S4 Conteggio = k+3 => S6
S6 - FINE	0	0	0	0	0	0	0	1	0	0	L_START = 0	S0

i. FSM: DESCRIZIONE DEGLI STATI

• Stato S0 (Stato iniziale)

All'inizio, la FSM si trova nello stato S0. In questo stato, la macchina attende che il segnale `i_start` sia attivato.

Se `i_start` è alto, la FSM transita nello stato S1. In caso contrario, rimane nello stato S0.

Le azioni eseguite in questo stato sono limitate: i segnali di uscita come `o_mem_addr`, `o_mem_en`, `o_mem_data`, `o_sr_enable`, e altri sono tutti impostati su valori di "default" (in genere 0), indicando che nessuna elaborazione è in corso. Il contatore `o_rstcounter` è abilitato a "1", resettando il contatore.

• Stato S1 (Lettura dei Comandi)

Nel secondo stato, S1, la FSM inizia la lettura dei comandi in ingresso dalla memoria.

La macchina calcola l'indirizzo di memoria come la somma di `i_add` e `i conteggio`, scorrendo così nella memoria. In questa fase, la memoria è abilitata (`o_mem_en = 1`), ma non viene scritto nulla nella stessa.

Inoltre, viene attivato il segnale `o_sr_enable` per abilitare la logica di shift e permettergli di immagazzinare i comandi.

• Stato S2 (Lettura dei k dati che andranno nel buffer)

Nel passaggio a S2, la FSM confronta il valore di `i conteggio` con una soglia. Quando `i conteggio` raggiunge il valore $17 + i_k$, la macchina prosegue verso il prossimo stato (S3). Durante questo stato, la memoria rimane abilitata per poter leggere i k byte da elaborare e salvarli nel buffer.

• Stato S3 (Reset del contatore e Preparazione del filtro)

Nel terzo stato (S3), la FSM esegue il RESET del contatore, quindi non attiva né la memoria né altre azioni. La macchina si prepara a passare a S4 e tutti gli altri segnali sono disabilitati.

• Stato S4 (Esecuzione Filtro)

Quando la FSM raggiunge lo stato S4, il segnale `o_fd_enable` viene attivato, abilitando il filtro per l'elaborazione. Tuttavia, in questo stato la macchina non effettua alcuna operazione di memoria (in parallelo il filtro sta elaborando il dato) e si prepara a passare allo stato S5.

• Stato S5 (Scrittura in Memoria)

La FSM scrive effettivamente nella memoria. Quando `i conteggio` è maggiore di 4, la macchina abilita la scrittura in memoria (`o_mem_we = 1`) e il dato da scrivere è `i_out_elaborator`.

Inoltre, l'elaborazione del contatore è attivata (`o_count_enable = 1`). Se `i conteggio` raggiunge il valore $i_k + 3$, la FSM passa allo stato S6.

Durante alcune operazioni, la nostra macchina perde 4 cicli di clock all'inizio dell'elaborazione, senza accumularli nel tempo. Per compensare questo ritardo nella scrittura in memoria, l'indirizzo `o_mem_addr` viene calcolato come $I_ADD + K + 17 + CONTEGGIO - 4$.

Il termine -4 serve a correggere lo slittamento causato dalla perdita iniziale di cicli, evitando di sovrascrivere dati indesiderati. Per garantire che la memoria venga modificata solo al momento giusto, il segnale di scrittura `o_mem_we` viene attivato (posto a 1) solo quando il contatore (counter) ha raggiunto il valore 4 dall'inizio della produzione dei dati.

• Stato S6 (Fine dell'Elaborazione)

Nell'ultimo stato, S6, la macchina segnala che l'elaborazione è terminata attraverso l'attivazione del segnale `o_done`. Tutti gli altri segnali vengono disattivati e la FSM ritorna allo stato S0 quando `i_start` è basso.

ii. FSM: DESCRIZIONE DEI SEGNALI

component FSM is

port(

i_mem_data: in std_logic_vector(7 downto 0);

segnale tramite cui si accede ai dati esposti dalla memoria.

i_k: in std_logic_vector(15 downto 0);

rappresenta il vettore, uscente dallo *SHIFT REGISTER*, che permette di ottenere la lunghezza della sequenza in input.

i_out_elaborator: in std_logic_vector(7 downto 0);

Input che arriva dal *FILTRO DIFFERENZIALE*, contiene, ad ogni step, il risultato calcolato dal filtro, pronto ad essere inserito in memoria.

i_start: in std_logic;

i_add: in std_logic_vector(15 downto 0);

rappresenta l'indirizzo di memoria da cui partire ad elaborare i segnali.

i_clk: in std_logic;

i_rst: in std_logic;

i conteggio: in std_logic_vector(23 downto 0);

segnale di conteggio che arriva dal *CONTATORE*.

o_rstcounter: out std_logic;

segnale interno va al *CONTATORE*. Si occupa di resettarlo e prepararlo a nuovi conteggi.

o_count_enable: out std_logic;

abilitazione *CONTATORE*.

o_sr_enable: out std_logic;

abilitazione *SHIFT REGISTER*.

o_mem_addr: out std_logic_vector(15 downto 0);

segnala alla memoria a quale address vogliamo accedere.

o_mem_data: out std_logic_vector(7 downto 0);

dato che vogliamo scrivere in memoria.

o_mem_we: out std_logic;

abilitazione della memoria in scrittura.

o_mem_en: out std_logic;

abilitazione memoria.

o_done: out std_logic;

o_fd_enable: out std_logic;

abilitazione *FILTRO DIFFERENZIALE*.

o_buf_enable: out std_logic

abilitazione *BUFFER*.

);

end component FSM;

d. SHIFT REGISTER

All'inizio dell'elaborazione, il sistema ha bisogno di raccogliere alcune informazioni fondamentali dalla memoria, come la lunghezza della sequenza e i coefficienti del filtro, contenute nei primi 17 byte. Questo compito viene svolto dal **registro a scorrimento (Shift Register)**, che legge e organizza i dati iniziali.

Ad ogni ciclo di clock, se lo shift register è abilitato ($i_sr_enable = 1$) il registro riceve un byte dalla memoria e lo sposta verso sinistra, fino a riempire tutti i campi necessari. Una volta che i dati sono stati completamente caricati, vengono suddivisi in tre parti:

- **o_k**, che indica la lunghezza della sequenza di dati da convertire col filtro.
- **o_opmode**, che specifica quale filtro utilizzare.
- **o_coeff**, che contiene i coefficienti del filtro.

Grazie a questo modulo, i dati di configurazione vengono raccolti e distribuiti in modo ordinato, permettendo alla FSM di impostare correttamente il sistema.

i. SHIFT REGISTER: DESCRIZIONE DEI SEGNALI

```
component SHIFT_REG is
  port(
    i_clk, i_rst, i_sr_enable: in std_logic;
    i_mem_data: in std_logic_vector(7 downto 0);
    -- segnale di input che si occupa di portare i dati dalla memoria.
    o_k: out std_logic_vector(15 downto 0);
    -- rappresenta il numero dei dati da convertire col filtro.
    o_coeff: out std_logic_vector(111 downto 0);
    -- esposizione coefficienti per filtro.
    o_opmode: out std_logic_vector(7 downto 0)
    -- selezione del modulo da utilizzare.
  );
end component SHIFT_REG;
```

e. CONTATORE

Per sapere a che punto dell'elaborazione ci si trova, il sistema utilizza un **contatore**, che tiene traccia del numero di cicli di elaborazione, fornendo un riferimento temporale per l'esecuzione delle operazioni. La FSM infatti, sfrutta il contatore per sapere quando passare alla fase successiva e per capire quando:

- Sono stati letti tutti i dati necessari;
- Il filtro ha terminato l'elaborazione;
- Tutti i dati filtrati sono stati scritti in memoria.

Iniziando da zero, il contatore viene incrementato ad ogni ciclo di clock se abilitato (`o_count_enable`) e viene azzerato all'inizio di una nuova elaborazione tramite il segnale di RESET (`o_rstcounter`).

Quando il contatore raggiunge il valore **K + 3**, significa che il processo è terminato e la FSM attiva il segnale **DONE** per indicare il completamento dell'operazione.

i. CONTATORE : DESCRIZIONE DEI SEGNALI

component COUNTER is

```
port(  
  i_clk, i_rst, i_count_enable: in std_logic;  
  o conteggio: out std_logic_vector(23 downto 0)
```

Il contatore ha come unico segnale di output un conteggio di 24 bit, valore scelto poiché nella fase iniziale di conteggio deve poter registrare k+17 cicli, essendo che k può arrivare fino a due byte, per contare k+17 servono un minimo di 3 byte.

```
);
```

```
end component COUNTER;
```

f. BUFFER

Il filtro differenziale lavora su una finestra di **sette valori consecutivi**, quindi è fondamentale avere un modulo che organizzi i dati in questa struttura. Questo compito è svolto dal **buffer**, che memorizza temporaneamente i dati letti dalla memoria e li mette a disposizione del filtro.

Il buffer espone una **finestra scorrevole**: allo stato S2 tutti i dati vengono immagazzinati nel buffer e successivamente, un ciclo di elaborazione alla volta, esso espone il byte corrispondente al dato da elaborare come centro della finestra, con i dati precedenti e quelli successivi nelle rispettive posizioni. L'obiettivo è far sì che per ciascuno dei k dati da convertire presenti in memoria, il buffer presenti una finestra al differential filter con quel dato al centro e quelli a lui vicini nelle posizioni rispettive all'interno della finestra. Se la sequenza è troppo corta e non ci sono abbastanza valori disponibili, il buffer inserisce **zeri** per riempire la finestra. Questo accade all'inizio e alla fine della sequenza, dove non ci sono abbastanza dati precedenti o successivi per completare i sette elementi.

i. BUFFER: DESCRIZIONE DEI SEGNALI

component buf is

```
port(  
  i conteggio: in std_logic_vector(23 downto 0);  
  Ingresso per tenere conto del conteggio (e quindi di quale dei k dati si sta analizzando).  
  i_mem_data: in std_logic_vector(7 downto 0);  
  dati provenienti dalla memoria.  
  i_buf_enable: in std_logic;
```

abilitatore del buffer dalla FSM. L'abilitazione è solo in lettura, perché per tutti gli stati in cui non sono previste operazioni di interazione BUFFER-DF, questo stampa valori di default che non vengono letti dal DIFFERENTIAL FILTER in quanto disabilitato.

```
i_k: in std_logic_vector(15 downto 0);  
    numero di parole da contenere, dato proveniente dallo SHIFT REGISTER.  
i_clk, i_rst: in std_logic;  
o_data0: out std_logic_vector(7 downto 0);  
o_data1: out std_logic_vector(7 downto 0);  
o_data2: out std_logic_vector(7 downto 0);  
o_data3: out std_logic_vector(7 downto 0);  
o_data4: out std_logic_vector(7 downto 0);  
o_data5: out std_logic_vector(7 downto 0);  
o_data6: out std_logic_vector(7 downto 0)  
    byte costituenti la finestra scorrevole esposta al DIFFERENTIAL FILTER.  
);  
end component;
```

g. FILTRO DIFFERENZIALE

Il **filtro differenziale** è il modulo responsabile dell'elaborazione numerica dei dati. Il suo compito è quello di prendere una finestra di sette valori consecutivi e calcolare un nuovo valore filtrato, combinando i dati con un insieme di coefficienti predefiniti.

Il sistema supporta due tipi di filtro:

- **Filtro di ordine 3**, che utilizza i primi 7 coefficienti della sequenza C1-C14 presente in memoria e ha una normalizzazione basata su $n = 12$;
- **Filtro di ordine 5**, che utilizza gli ultimi 7 coefficienti e una normalizzazione con $n = 60$.

Esso dunque si occupa della **normalizzazione**, che avviene tramite una serie di **shift a destra**. Questi shift sono equivalenti a divisioni per potenze di due e permettono di ridurre il valore in maniera controllata. Il tutto avviene seguendo le regole descritte nel primo capitolo, garantendo integrazione e correzione di dati persi tramite troncamento durante gli shift di numeri negativi. Inoltre esso si occupa di gestire risultati che sarebbero maggiori di 127 o minori di -128, ma non rappresentabili con un byte in complemento a due, **saturando** l'uscita ai rispettivi valori limite.

i. FILTRO DIFFERENZIALE: DESCRIZIONE DEI SEGNALI

component DIFFERENTIAL_FILTER is

```
port(  
    i_conteggio: in std_logic_vector(23 downto 0); <- conteggio proveniente dal COUNTER.  
    i_data0: in std_logic_vector(7 downto 0); <- finestra di 7 byte esposta dal BUFFER.  
    i_data1: in std_logic_vector(7 downto 0);  
    i_data2: in std_logic_vector(7 downto 0);  
    i_data3: in std_logic_vector(7 downto 0);  
    i_data4: in std_logic_vector(7 downto 0);  
    i_data5: in std_logic_vector(7 downto 0);  
    i_data6: in std_logic_vector(7 downto 0);  
    i_fdenable: in std_logic; <- abilitazione all'elaborazione e alla produzione di output da parte di FSM.  
    i_opmode: in std_logic_vector(7 downto 0); <- indicazioni sui coefficienti da prelevare, provenienti  
        dallo SHIFT REGISTER.  
    i_coeff: in std_logic_vector(111 downto 0); <- coefficienti esposti dallo SHIFT REGISTER.  
    i_k: in std_logic_vector(15 downto 0); <- numero di finestre da elaborare, esposto da SHIFT REGISTER.  
    i_clk, i_rst: in std_logic;  
    o_fd_out: out std_logic_vector(7 downto 0) <- dato calcolato, inviato alla FSM.  
);  
end component;
```

h. ANALISI DELLE FASI DI PROGETTAZIONE

Il progetto è stato sviluppato seguendo un approccio incrementale, suddiviso in più fasi.

Nella fase iniziale, è stata effettuata una disamina approfondita della specifica, seguita da una sessione di brainstorming tra i membri del gruppo per valutare possibili soluzioni al problema proposto.

Successivamente, si è passati alla fase di progettazione del componente, condotta inizialmente su carta. Il lavoro è partito da una visione ad alto livello del sistema, per poi scendere progressivamente nel dettaglio, definendo il comportamento di ogni componente e le relative interconnessioni.

Una volta delineata l'architettura generale, è iniziata la fase di implementazione in VHDL, con lo sviluppo e la verifica delle singole componenti.

Durante questo processo, il progetto ha attraversato sei revisioni principali, ognuna delle quali ha introdotto miglioramenti e correzioni:

1. **Implementazione generale:** prima versione con una struttura di base del componente;
2. **Correzioni di calcolo e gestione della FSM:** l'analisi dei test eseguiti, supportata dal tool Vivado, ha evidenziato alcune criticità, risolte mediante debugging dei segnali, visualizzati grazie al tool e validazione con test su carta;
3. **Correzioni di normalizzazione:** risoluzione di comportamenti inattesi emersi in alcuni casi di test;
4. **Aggiunta della saturazione ai valori limite:** implementazione di un meccanismo per vincolare i risultati entro i range previsti, a seguito di una nuova release della specifica;
5. **Flessibilità nei coefficienti dei filtri di ordine 3 e 5:** aggiornamento del comportamento della macchina per supportare valori non costanti per i coefficienti C1- C14;
6. **Ottimizzazione della gestione di START e RESET multipli:** miglioramenti per garantire un funzionamento corretto anche in presenza di ripetuti comandi di START e RESET.

Questa metodologia iterativa ha permesso di affinare progressivamente il progetto, garantendo una soluzione robusta e aderente alle specifiche richieste.

3. RISULTATI SPERIMENTALI

a. SINTESI

i. ANALISI DEL TIMING REPORT

L'analisi del Timing Report evidenzia il margine di sicurezza temporale (Slack) e i principali contributi al ritardo nel circuito sintetizzato in VHDL.

Parametro	Valore	Descrizione
Slack (MET)	9.654 ns	Margine positivo: il circuito rispetta i vincoli di timing con un buon margine.
Periodo di Clock	20.000 ns	Frequenza operativa: 50 MHz.
Ritardo Totale	10.228 ns	Tempo impiegato dal segnale per propagarsi.
Ritardo Logico	6.613 ns (64.7%)	Dovuto alla computazione nei blocchi logici.
Ritardo Routing	3.615 ns (35.3%)	Tempo perso nella connessione tra i blocchi logici.
Skew del Clock	-0.145 ns	Differenza di arrivo del clock tra sorgente e destinazione.

Il percorso più critico è dominato dai seguenti elementi:

- Blocchi CARRY4: responsabili della propagazione del riporto nelle operazioni aritmetiche (11 livelli logici su 17);
- LUT (Look-Up Table): coinvolte nella computazione combinatoria;
- Registri FDRE e FDCE: sincronizzano i segnali col clock.

Il progetto soddisfa i requisiti temporali con un ampio margine, consentendo eventuali ottimizzazioni. L'analisi suggerisce che il ritardo è influenzato principalmente dalla propagazione del riporto nei CARRY4, che potrebbe essere migliorata con una diversa organizzazione delle operazioni aritmetiche.

3. RISULTATI SPERIMENTALI

ii. ANALISI DELLA LOGICA

La tabella sottostante riporta l'utilizzo delle risorse logiche Slice, tra cui Look-Up Tables (LUTs) e registri.

Tipologia	Usati	Fissi	Disponibili	Utilizzo (%)
Slice LUTs	146031	0	134600	108.49
- LUT come Logica	60015	0	134600	44.59
- LUT come Memoria	86016	0	46200	186.18
-- LUT come RAM Distribuita	86016	0	-	-
-- LUT come Registro Shift	0	0	-	-
Slice Registers	368	0	269200	0.14
- Registri come Flip Flop	368	0	269200	0.14
- Registri come Latch	0	0	269200	0.00
Multiplexer (Mux)				
- F7 Muxes	7616	0	67300	11.32
- F8 Muxes	3712	0	33650	11.03

Si noti la presenza di 0 registri come latch, che indicano l'assenza di elementi non sincroni nel circuito.

Totale	Clock Enable	Sincrono	Asincrono
0	–	-	-
0	–	-	Set
0	–	-	Reset
0	–	Set	-
0	–	Reset	-
0	Sì	-	-
0	Sì	-	Set
248	Sì	-	Reset
0	Sì	Set	-
120	Sì	Reset	-

Il progetto sfrutta intensamente le LUT come memoria distribuita, senza RAM dedicata o unità DSP. L'utilizzo di IO è minimo, e non vi sono blocchi logici avanzati impiegati.

3. RISULTATI SPERIMENTALI

b. SIMULAZIONI

La fase di simulazione è stata svolta dopo aver completamente realizzato il progetto.

All'inizio sono state testate le funzionalità generali e poi si è andati a ispezionare i corner cases.

Si procede con la disamina dei casi di test e l'analisi delle motivazioni dettagliate che hanno portato alla creazione degli stessi.

i. TEST FORNITI DAL DOCENTE

La fase di testing ha avuto inizio con la validazione del codice VHDL, eseguita mediante i test bench forniti insieme alla specifica del progetto. Questi test bench, predisposti dai docenti, sono stati considerati il punto di riferimento principale per la valutazione della correttezza e dell'affidabilità del codice.

Inizialmente, ogni versione del codice sviluppato è stata sottoposta all'esecuzione dei test forniti, permettendo di individuare eventuali discrepanze rispetto al comportamento atteso.

Questa prima fase ha consentito di effettuare un'analisi preliminare delle funzionalità implementate, nonché di identificare eventuali errori logici o sintattici nel codice.

Le attività di debugging sono state condotte in modo sistematico, procedendo con l'analisi dei risultati ottenuti a seguito dell'esecuzione dei test.

In particolare, si è spesso proceduto con l'analisi dei singoli segnali emessi dalle varie componenti tramite l'utilizzo del tool VIVADO, verificando che il comportamento fosse conforme alle specifiche di progetto e all'andamento atteso, analizzandolo dall'EXCEL per generare i test e da controlli manuali dei calcoli.

Nel caso in cui venissero riscontrate anomalie, si è provveduto a una revisione del codice, con successive iterazioni di test fino a ottenere il corretto funzionamento del sistema.

Questa metodologia di validazione ha garantito un approccio rigoroso nella verifica del codice VHDL, permettendo di individuare e correggere tempestivamente eventuali errori, nonché di affinare progressivamente la nostra implementazione fino a raggiungere un livello di affidabilità e correttezza soddisfacente.

ii. MINIMO NUMERO DI DATI IN INPUT

Il test eseguito ha avuto l'obiettivo di analizzare il comportamento della macchina quando sottoposta al valore minimo sensato di dati in ingresso, corrispondente a sette parole da 8 bit, come indicato dalla specifica.

Sulla base di questa osservazione, è stato sviluppato un test bench specifico per verificare il corretto funzionamento del sistema in tale condizione.

Lo stesso caso di test è stato successivamente applicato ai filtri modulo 3 e modulo 5, al fine di garantire la coerenza del comportamento dell'intero sistema anche per queste configurazioni. Per l'esecuzione di questa fase di test, i coefficienti utilizzati sono stati quelli forniti come riferimento all'interno della specifica del progetto.

Tipo di Filtro: sono stati testati entrambi i filtri

C1-C14 : 0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1

In mod3: 51, 68, -26, 25, 64, 14, -125

In mod5 : -3, 111, 84, -36, -67, 28, 34

Out mod3 : -45, 52, 28, -63, 0, 121, 3

Out mod5: -69, -69, 98, 98, -53, -68, 29

3. RISULTATI SPERIMENTALI

iii. MASSIMO NUMERO DI DATI IN INPUT

In opposizione al caso minimo, è stato anche testato il massimo numero di dati in input disponibili. Per la creazione di questo caso di test si è partiti da calcoli manuali, per trovare la dimensione massima dei dati in input.

Sapendo che la memoria è indicizzata a 2 byte (16 bit), possiamo calcolare il massimo numero esprimibile in 16 bit.

$$1111.1111.1111.1111 \text{ bin} = 65.535 \text{ dec.}$$

65.535 è il numero massimo di dati inseribile in memoria.

Si è proceduto poi col rimuovere l'offset dei coefficienti che vengono sempre passati come primi in ingresso

$$65.535 - 17 = 65.518$$

andando poi a dividere per due e trovare quindi il numero massimo per saturare la memoria in input

$$65.518 / 2 = 32.759$$

Si è quindi creato un testcase che avesse esattamente 32.759 numeri in input e si è verificato che l'output fosse corretto e che non ci fossero accessi in memoria fuori dai boundaries.

Lo stesso test è stato svolto sia per il filtro modulo 3 che per il filtro modulo 5.

iv. MASSIMO NUMERO DI DATI IN INPUT +1

Per confermare che il limite massimo individuato fosse effettivamente il valore oltre il quale si verificano errori di accesso in memoria, è stato eseguito un ulteriore test con un numero di input pari a 32.760. L'obiettivo era verificare il comportamento del sistema in presenza di un overflow della memoria. Il risultato ha confermato l'ipotesi iniziale, mostrando un comportamento anomalo che ha validato il limite massimo di dati gestibili.

ERROR: Index 65536 out of bound 65535 downto 0

v. 0 PRE E POST NORMALIZZAZIONE

Sono stati condotti dei test portando la somma parziale pre-normalizzazione ad assumere valori molto bassi che, una volta normalizzati, andassero a 0. Sono stati condotti casi di test anche per quelle somme che già prima della normalizzazione fossero uguali a 0. Ciò perchè è stata introdotta una correzione di normalizzazione per tutti i numeri negativi, per cui è utile controllare cosa accade con calcoli al limite del positivo, 0 appunto.

Tipo di Filtro: sono stati testati entrambi i filtri

C1-C14 : 0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1

In mod3: -37, -94, 44, 93, -73, -10, 0, 0, 0, 40, 0, 0, 0, 121, 0, 1, 72, 82, 73

In mod5 : 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0

Out mod3 : 64, -44, -124, 81, 63, -54, 0, 3, -24, 0, 26, 6, -78, 0, 85, -49, -45, 0, 47

Out mod5: 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

3. RISULTATI SPERIMENTALI

vi. TUTTI 0

Un'altra condizione di test importante ha riguardato l'inserimento di una sequenza composta interamente da zeri. Sono stati realizzati due test bench per verificare la risposta del sistema a questa casistica, analizzando l'output e confrontandolo con i valori attesi. Gli zeri sono dati particolarmente interessanti poiché, dovendo gestire shift e normalizzazioni (dunque divisioni e moltiplicazioni). Errare un calcolo con 0 potrebbe portare ad errori importanti, ragion per cui abbiamo provato a massimizzare il numero di 0 in sequenza per garantire che non ci fossero conflitti.

Tipo di Filtro: sono stati testati entrambi i filtri
C1-C14 : 0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1
In mod3 e mod5: 0, 0, 0, 0, 0, 0, 0, 0
Out mod3 e mod5: 0, 0, 0, 0, 0, 0, 0, 0

vii. COSTANTE

Per analizzare il comportamento del modulo in presenza di un valore costante, è stato eseguito un test specifico con un valore di ingresso pari a -128. L'obiettivo era valutare il mantenimento della costanza del segnale e il corretto trattamento della normalizzazione e della saturazione. Anche in questo caso, il test è stato ripetuto per entrambi i filtri. Abbiamo notato che, correttamente, i valori iniziali e finali di output sono diversi da zero (in quanto calcolati con 0 di riempimento per i corner values), successivamente, non appena la finestra di calcolo si stabilizza su tutti valori costanti, utilizzando i coefficienti presenti in specifica si ottiene un bilanciamento perfetto e un output pari a 0. Questo comportamento ovviamente non è atteso con coefficienti mutati e non bilanciati.

Tipo di Filtro: sono stati testati entrambi i filtri
C1-C14 : 0, -1, 8, 0, -8, 1, 0, 1, -9, 45, 0, -45, 9, -1
In mod3: -128, -128, -128, -128, -128, -128, -128, -128
In mod5 : -128, -128, -128, -128, -128, -128, -128, -128
Out mod3 : 73, -8, 0, 0, 0, 0, 10, -71
Out mod5: 78, -15, 2, 0, 0, -1, 17, -77

viii. VALORE MASSIMO E MINIMO IN INPUT

Ulteriori verifiche sono state condotte utilizzando il valore massimo e minimo rappresentabili in complemento a 2 su 8 bit (+127 e -128), per garantire che il modulo gestisse correttamente i casi limite senza introdurre errori di overflow o underflow.

```
-128, -128, -128, -128, -128, -128, -128, 0, 0, 0, 0, -45, 9, -1      -- C1-C14, S=0  
);
```

```
signal scenario_input : scenario_type := (-128, -128, -128, -128, -128, -128, -128, -128);  
signal scenario_output : scenario_type := (127, 127, 127, 127, 127, 127, 127, 127);
```

I seguenti valori sono stati scelti poiché vanno a generare, al centro dell'elaborazione, il massimo valore possibile esprimibile, infatti $(-128) * (-128) * 7 = 114.688$ rappresenta il caso pessimo.

```
0, -1, 8, 0, -8, 1, 0, -128, -128, -128, -128, -128, -128      -- C1-C14, S=1  
signal scenario_input : scenario_type := (127, 127, 127, 127, 127, 127, 127, 127);  
signal scenario_output : scenario_type := (-128, -128, -128, -128, -128, -128, -128, -128);
```

Questi valori invece generano il minimo valore esprimibile, infatti $(-128) * 127 * 7 = -113.792$.

I casi sopra mostrati vengono correttamente gestiti dal componente, in quanto per i valori dei risultati provvisori ottenuti tramite moltiplicazione con coefficienti e per immagazzinare la somma massima o minima possibile, prima della normalizzazione, i cui valori sono stati esposti nell'esempio, sono stati utilizzati 3 byte, così da evitare l'overflow.

3. RISULTATI SPERIMENTALI

ix. START MULTIPLI

Il comportamento del modulo in risposta a più segnali di START consecutivi è stato verificato attraverso un test che simulasse richieste ripetute di elaborazione. L'obiettivo era assicurarsi che il sistema non entrasse in stati incoerenti e che il segnale DONE fosse correttamente gestito. Sono stati alzati 3 segnali di START in totale che proponessero 3 configurazioni e dati da elaborare differenti in due test bench diversi: il primo elabora zone differenti di memoria, il secondo, continua a sovrascrivere il medesimo indirizzo di memoria con nuovi dati.

x. RESET

La gestione del segnale di RESET è stata verificata attraverso simulazioni che valutassero il corretto ripristino dello stato iniziale del modulo. Il test ha confermato che, a seguito di un RESET, il sistema tornasse a uno stato pronto per ricevere un nuovo comando START.

Si è inoltre posta attenzione a far sì che ci fosse un segnale di RESET in corrispondenza di ogni stato della FSM per verificare la correttezza generale del componente.

xi. TEST DI ROBUSTEZZA E DI CORRETTEZZA DI ACCESSO ALLA MEMORIA

Sono stati creati con il file excel fornito insieme alla specifica e sottoposti al componente, 50 test casuali, con l'obiettivo di verificare la correttezza e robustezza del codice.

I casi di test sono stati divisi in modo uniforme tra i diversi tipi di filtri, garantendo uniformità nel test. Sono inoltre stati aggiunti test volutamente brevi per verificare che non si andasse a sovrascrivere memoria non necessaria, soprattutto per gli ultimi elementi della sequenza di k valori da elaborare, per valutare l'efficienza della correzione su un ritardo di 4 cicli riscontrato in precedenza. Ci siamo assicurati che non ci siano ritardi in output e che non avvengano accessi fuori dagli address desiderati.

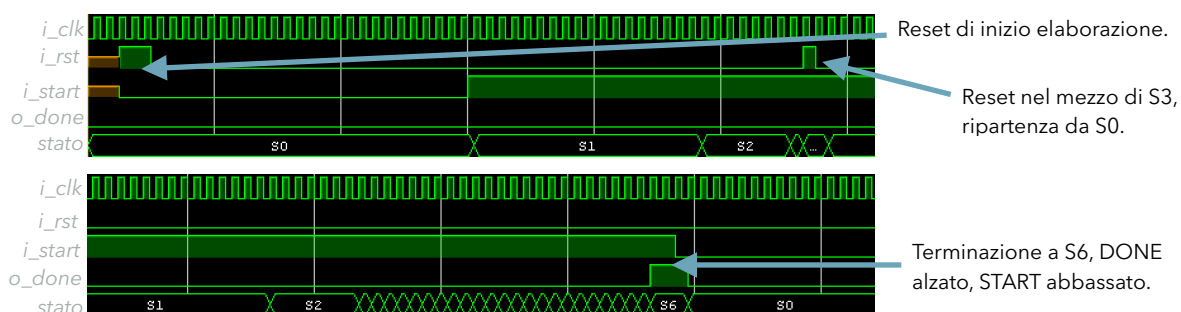
xii. TEST CON COEFFICIENTI C1- C14 CASUALI

Per analizzare la robustezza del sistema rispetto a coefficienti di filtro differenti da quelli standard, è stato eseguito un test con valori casuali per i coefficienti C1-C14.

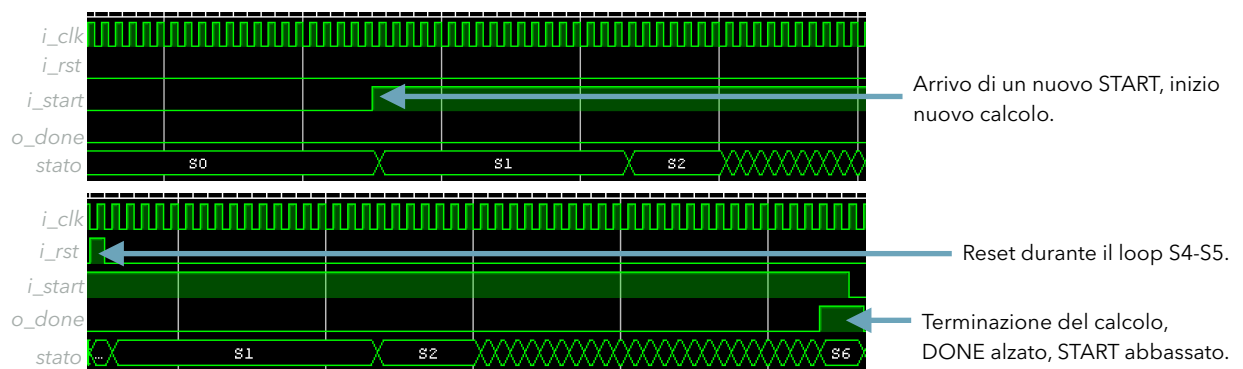
L'obiettivo è verificare la stabilità dell'algorithm di filtraggio e la gestione della normalizzazione in condizioni non previste dai coefficienti standard.

xiii. START E RESET COMBINATI

Infine, è stato eseguito un test con una sequenza di START e RESET alternati per verificare che il modulo gestisse correttamente l'inizializzazione e non producesse comportamenti indesiderati.



3. RISULTATI SPERIMENTALI



c. CONCLUSIONI DELLA FASE DI SIMULAZIONE

L'approccio sistematico adottato per la simulazione e il testing ha consentito di validare il corretto funzionamento del modulo, individuando e correggendo eventuali anomalie. L'uso di test specifici per casi limite e condizioni particolari ha garantito la robustezza dell'implementazione e la conformità alle specifiche del progetto.

4. CONCLUSIONI

Il progetto di reti logiche sviluppato ha permesso di realizzare un modulo hardware in grado di interfacciarsi con una memoria esterna, leggere e processare dati applicando un filtro differenziale, e scrivere i risultati nella stessa memoria.

L'implementazione ha seguito rigorosamente la specifica fornita, garantendo la corretta gestione della memoria e l'elaborazione dei dati secondo le modalità previste.

L'architettura progettata è stata suddivisa in moduli ben definiti, tra cui una FSM per il controllo del flusso delle operazioni, uno shift register per l'immagazzinamento delle istruzioni, un buffer per la gestione dei dati, un contatore per il tracciamento delle operazioni e un filtro differenziale per l'elaborazione numerica. Questa suddivisione ha permesso di ottenere una progettazione modulare, facilitando debugging e ottimizzazione del sistema.

Durante la fase di verifica, il modulo è stato testato con diverse configurazioni di input per valutare il corretto funzionamento del filtro nelle varie condizioni. I risultati ottenuti hanno confermato la robustezza del sistema e la sua conformità alla specifica, evidenziando la corretta gestione dei dati ai corner cases e l'applicazione dei coefficienti di normalizzazione.

In conclusione, il progetto ha rappresentato un'esperienza significativa nell'ambito della progettazione di sistemi digitali, fornendo competenze pratiche nell'uso del linguaggio VHDL, nella gestione della memoria e nella progettazione di architetture hardware complesse. Le soluzioni adottate hanno permesso di realizzare un modulo efficiente e conforme alle richieste, dimostrando l'importanza di un approccio strutturato alla progettazione logica e all'ottimizzazione delle risorse hardware.